



FACULTAD DE INGENIERÍA

CARRERA DE INGENIERÍA DE SISTEMAS COMPUTACIONALES

INFORME DE PROYECTO

TÍTULO DESAFIO: SistemaRural-PE

Diseño de software multiparadigma para la gestión del Puesto de
Salud Chontapaccha (Cajamarca)

Autores:

Francisco Enrique Chávez Portal — N00553501

Alex Rodrigo Delgado Romero — N00538607

Evelin Noemi Escobar Huamán — N00536771

Brayan Erick Figueroa Chaupe — N00536285

Ana Raquel Olortegui Rodríguez — N00536740

Curso:

Lenguajes de Programación — UAIN1288P

Docente:

Ronald Ivan Alcantara Quispe

Cajamarca – Perú 2026

ÍNDICE

Caràtula.....	1
1. Definición y delimitación del reto	3
1.1 Establecimiento seleccionado	3
1.2 Requerimientos de software identificados.....	3
1.3 Restricciones de diseño.....	4
1.4 Criterios de éxito medibles y verificables	4
2. Planificación y ruta de trabajo del equipo	4
3. Fundamentación del enfoque de diseño y selección de paradigmas.....	6
4. Validación de viabilidad del componente técnico más incierto.....	7
5. Diagnóstico de riesgos y autorregulación	8
Referencias	10

1. Definición y delimitación del reto

1.1 Establecimiento seleccionado

El equipo seleccionó el Puesto de Salud Chontapaccha, ubicado en el distrito y provincia de Cajamarca, departamento de Cajamarca, como caso aplicado para el presente proyecto. Su existencia y registro se confirma en un documento oficial del Gobierno Regional de Cajamarca sobre titulación de tierras de establecimientos de salud, donde figura con un área registrada de 104.24 m² (Gobierno Regional Cajamarca, 2013).

El equipo cursó una solicitud formal (Oficio DA/SIC-00377-2026) para realizar una visita de campo y una entrevista con el personal responsable, la cual se encuentra en trámite hasta la fecha de entrega de esta Evaluación Parcial. Por este motivo, la caracterización del establecimiento y de su problemática se realizó con fuentes públicas oficiales, mientras se espera la autorización. Esta decisión se documenta como parte del diagnóstico de riesgos en la sección 5.

Por su categoría (Puesto de Salud), el establecimiento corresponde al primer nivel de atención (I-1), el mismo segmento que evidencia mayores brechas de digitalización e infraestructura en el país. Según un informe de Videnza Instituto y Videnza Consultores, el 96% de los establecimientos de este nivel no cuenta con capacidad instalada adecuada, y el 76% no tiene capacidad diagnóstica básica (Gestión, s.f.-a). De forma similar, un documento técnico del MINSA sobre brechas del sector salud reportó que, a enero de 2020, el 77.8% de los establecimientos de primer nivel de atención a nivel nacional presentaba capacidad instalada inadecuada (Gestión, s.f.-b). El propio desafío del curso, citando al MINSA (2023), señala que más del 60% de los establecimientos I-1 e I-2 de zonas rurales de Cajamarca, Puno y Loreto carecen de sistemas informáticos para registrar atenciones, gestionar inventario y monitorear citas (UPN, 2026). Chontapaccha, por su categoría y ubicación, se enmarca razonablemente en este mismo universo de establecimientos.

1.2 Requerimientos de software identificados

- RQ-01: Registrar pacientes con sus datos personales básicos y un identificador único (DNI).
- RQ-02: Registrar y consultar citas médicas por paciente y por fecha.

- RQ-03: Gestionar el inventario de medicamentos e insumos (existencias, entradas y salidas).
- RQ-04: Validar automáticamente el RUC de proveedores de insumos y medicamentos.
- RQ-05: Generar reportes periódicos de atenciones para su envío a la DIRESA Cajamarca.
- RQ-06: Filtrar y buscar pacientes o citas mediante criterios simples (nombre, DNI, fecha).
- RQ-07: Tratar los datos personales de pacientes de forma segura, sin exponerlos en texto plano, conforme a la Ley N.º 29733.

1.3 Restricciones de diseño

- RD-01: El sistema debe construirse íntegramente con herramientas de código abierto (Python 3.x, VS Code, Git/GitHub).
- RD-02: El almacenamiento y tratamiento de datos personales debe cumplir la Ley N.º 29733, Ley de Protección de Datos Personales (Congreso de la República del Perú, 2011) — sin datos sensibles en texto plano.
- RD-03: El sistema debe funcionar de forma local, sin depender de una conexión a internet permanente, dado el contexto rural del establecimiento.
- RD-04: La interfaz y los requerimientos de hardware deben ser mínimos, asumiendo que el establecimiento cuenta con equipos de cómputo limitados.
- RD-05: El diseño debe alinearse con los lineamientos de la Política Nacional de Salud Digital del Perú (MINSA, 2022).

1.4 Criterios de éxito medibles y verificables

- CE-01: El registro de una nueva atención se completa en menos de 2 segundos de procesamiento.
- CE-02: La validación automática de RUC de proveedores identifica correctamente el 100% de los casos de la batería de prueba (15/15, ver sección 4).
- CE-03: El sistema impide el registro de pacientes duplicados mediante validación de DNI en el 100% de los casos probados.
- CE-04: El reporte de atenciones para la DIRESA se genera en menos de 5 segundos para un mes calendario de registros.

2. Planificación y ruta de trabajo del equipo

La siguiente tabla presenta la distribución de roles propuesta por el equipo. Los roles quedan sujetos a ajuste según la carga de trabajo real de cada hito.

Tabla 1. Roles y responsabilidades del equipo

Integrante	Rol propuesto	Responsabilidad principal
Ana Raquel Olortegui Rodríguez	Coordinadora general	Documentación, seguimiento del cronograma
Francisco Enrique Chávez Portal	Diseñador de clases	Modelado UML y patrones de diseño
Alex Rodrigo Delgado Romero	Programador de lógica	Implementación del módulo en Python
Evelin Noemi Escobar Huamán	Encargada de pruebas	Casos de prueba, validación, QA
Brayan Erick Figueroa Chaupe	Integrador	Repositorio Git/GitHub, control de versiones

Tabla 2. Cronograma de hitos

Fecha	Hito	Responsable	Estado
Sem. previa	H1: Prueba de viabilidad del componente incierto (AFND_RUC)	Alex Delgado	Completado
24 set.	H2: Definición de requerimientos, restricciones y criterios de éxito (esta EP)	Ana Olortegui	Completado
26 set.	H3: Diagrama de clases UML preliminar (depende de H2)	Francisco Chávez	Próximo
30 set.	H4: Creación y configuración del repositorio GitHub	Brayan Figueroa	Próximo

03 oct.	H5: Implementación del módulo funcional en Python (depende de H3)	Alex Delgado / Evelin Escobar	Próximo
08 oct.	H6: Pruebas automatizadas y revisión de calidad de código (depende de H5)	Evelin Escobar	Próximo

Ruta crítica: H3 depende de H2 (el modelado UML requiere los requerimientos definidos); H5 depende de H3 (la implementación requiere el diagrama de clases aprobado); H6 depende de H5 (las pruebas requieren código funcional). Un retraso en H2 o H3 impacta directamente la fecha de entrega de la EF.

Contingencia principal: si la autorización de visita a Chontapaccha no se resuelve antes de la entrega final, el equipo continuará con la caracterización basada en fuentes públicas oficiales (RENIPRESS, documentos del Gobierno Regional de Cajamarca), documentando esto como limitación en las conclusiones del informe final, tal como ya se aplicó para esta Evaluación Parcial.

3. Fundamentación del enfoque de diseño y selección de paradigmas

Se compararon tres paradigmas de programación aplicables al sistema, evaluados contra cuatro criterios técnicos vinculados directamente a los requerimientos de la sección 1.2:

Tabla 3. Comparación de paradigmas de programación

Criterio técnico	Orientado a objetos	Funcional	Orientado a eventos
Modelado de entidades del dominio (RQ-01, RQ-02)	Alta: clases Paciente, Cita, Medicamento con encapsulamiento	Baja: no modela entidades con estado propio	Baja: solo modela interacción, no el dominio
Transformación de colecciones (RQ-03, RQ-06)	Media: requiere bucles explícitos	Alta: map/filter/reduce sobre listas de registros	No aplica directamente
Validación y reportes (RQ-04, RQ-05)	Media	Alta: composición de funciones para validar y agregar datos	No aplica directamente

Interacción con el usuario (interfaz del sistema)	Media: posible con listeners como métodos de clase	Baja: no es su fortaleza natural	Alta: listeners y handlers están diseñados para esto
---	--	----------------------------------	--

Justificación de la selección: el equipo integrará los tres paradigmas de forma complementaria. La Programación Orientada a Objetos estructura las entidades centrales del sistema (RQ-01, RQ-02, RQ-03) mediante clases con encapsulamiento. La Programación Funcional se aplicará sobre las colecciones de registros para los reportes y filtros (RQ-05, RQ-06), aprovechando funciones de orden superior para transformar datos sin efectos colaterales. La Programación Orientada a Eventos gobernará la interfaz gráfica, vinculando cada acción del usuario (registrar, buscar, generar reporte) con su manejador correspondiente. Esta decisión queda documentada como restricción de diseño trazable: el módulo funcional (sección 1.4 del informe final) deberá evidenciar los tres paradigmas integrados sobre las mismas estructuras de datos del sistema.

4. Validación de viabilidad del componente técnico más incierto

El equipo identificó la validación automática del RUC de proveedores (RQ-04) como el componente de mayor incertidumbre técnica del sistema, dado que el RUC peruano sigue un patrón estructural específico (11 dígitos, con prefijo 10 para persona natural o 20 para persona jurídica) que debía verificarse formalmente antes de comprometerse a una implementación.

Como paso previo de diseño, el equipo modeló el patrón del RUC como un Autómata Finito No Determinista (AFND_RUC.jff) en JFLAP, para verificar formalmente que la regla no presentaba ambigüedades antes de programarla. Sin embargo, la evidencia de viabilidad que se documenta a continuación consiste en código ejecutado en Python, tal como exige la rúbrica de este criterio.

Código 1. Script de validación de RUC (validar_ruc.py)

```
import re
PATRON_RUC = re.compile(r"^(10|20)\d{9}$")

def validar_ruc(ruc: str) -> bool:
    return bool(PATRON_RUC.match(ruc))
```

El script se ejecutó contra una batería de 15 casos de prueba (RUC válidos con prefijo 10 y 20, y RUC inválidos por prefijo incorrecto, longitud incorrecta, caracteres no numéricos, cadena vacía y espacios extra).

Figura 1. Resultado de la ejecución de validar_ruc.py

RUC	Esperado	Obtenido	Resultado

'10456789123'	True	True	OK
'20601234567'	True	True	OK
'10111222333'	True	True	OK
'20999888777'	True	True	OK
'10000000001'	True	True	OK
'20123456789'	True	True	OK
'30456789123'	False	False	OK
'15456789123'	False	False	OK
'1045678912'	False	False	OK
'104567891234'	False	False	OK
'1045A789123'	False	False	OK
"	False	False	OK
'20-601234567'	False	False	OK
'10456789123 '	False	False	OK
'00456789123'	False	False	OK

Resultado final: 15/15 casos correctos			

Veredicto de viabilidad: VIABLE. Los 15 casos de prueba arrojaron coincidencia exacta entre el resultado esperado y el resultado obtenido por el script, confirmando que la regla puede implementarse de forma confiable mediante una expresión regular en Python.

Decisión derivada y ajuste al plan: se incorporará una clase ValidadorRUC (basada en esta función) en el diagrama de clases UML (sección 1.3 del informe final), y se agregó al cronograma (Tabla 2, hito H5) la tarea específica de integrar esta validación al módulo funcional, en lugar de tratarla como un detalle menor de RQ-04. El script validar_ruc.py se incorporará al repositorio GitHub del equipo como parte de la evidencia técnica.

5. Diagnóstico de riesgos y autorregulación

Tabla 4. Riesgos identificados y mitigación

Riesgo	Impacto	Acción de mitigación
La autorización de visita a Chontapaccha no llega a tiempo	Retraso o imposibilidad de caracterizar el establecimiento con entrevista real	Caracterización con fuentes públicas oficiales (RENIPRESS, Gobierno Regional Cajamarca) mientras se espera respuesta
Ambigüedad entre los entregables de JFLAP (autómatas) y el módulo final en Python/Java	Posible trabajo no contabilizado o mal dirigido	Consultar con el docente el alcance exacto de cada entregable antes de la EF
Conectividad limitada en la zona rural del establecimiento	El sistema podría no funcionar en condiciones reales de uso	Diseñar el sistema para operar de forma local (RD-03), sin dependencia de internet
Exposición accidental de datos personales de pacientes	Incumplimiento de la Ley N.º 29733 y daño a los usuarios	Aplicar encapsulamiento y evitar texto plano desde el diseño de clases (RD-02)

Ciclo de retroalimentación 1 — pivote de establecimiento: ante la falta de respuesta de Chontapaccha, el equipo evaluó como alternativa el Hospital II-E Simón Bolívar de Cajamarca, dada su amplia información pública disponible. Tras un análisis comparativo, se determinó que su perfil (hospital de mayor complejidad, con quirófano y telesalud) no correspondía al caso de referencia del desafío (Puesto de Salud Chontapaccha, categoría I-1, sin sistema alguno). Se decidió retornar a Chontapaccha como caso principal, documentando la entrevista pendiente como limitación en lugar de forzar un cambio de establecimiento. Este ajuste queda evidenciado en el historial de conversación y decisiones del equipo previo a esta entrega.

Ciclo de retroalimentación 2 — validación del componente incierto: la prueba del AFND_RUC (sección 4) confirmó la viabilidad del enfoque de validación planteado inicialmente, y como resultado se ajustó el cronograma para incluir explícitamente esta tarea en el hito H5, evitando que quedara como un supuesto no verificado dentro del módulo funcional.

La evidencia verificable de ambos ciclos de retroalimentación (historial de cambios del repositorio del equipo) se presenta en el Anexo A.

Referencias

Congreso de la República del Perú. (2011). Ley N.º 29733, Ley de Protección de Datos Personales. El Peruano. <https://www.leyes.congreso.gob.pe/Documentos/Leyes/29733.pdf>

Gestión. (s.f.-a). Estado grave: el 76% de postas rurales no pueden diagnosticar ni lo básico. <https://gestion.pe/peru/estado-grave-el-76-de-postas-rurales-no-pueden-diagnosticar-ni-lo-basico-noticia/>

Gestión. (s.f.-b). MINSA: el 51% de hospitales a nivel nacional con equipamiento obsoleto. <https://gestion.pe/economia/minsa-el-51-de-hospitales-a-nivel-nacional-con-equipamiento-obsoleto-noticia/>

Gobierno Regional Cajamarca. (2013). Titulación de tierras: predios entregados a establecimientos de salud [documento institucional]. <https://portal.regioncajamarca.gob.pe/>

Ministerio de Salud del Perú [MINSA]. (2022). Política Nacional de Salud Digital del Perú. <https://www.gob.pe/minsa>

Universidad Privada del Norte [UPN]. (2026). Evaluación final - Desafío del curso: Lenguajes de Programación (UAIN1288P) [documento de cátedra].

ANEXOS:

Anexo A: Evidencia del repositorio GitHub

Repositorio del equipo: <https://github.com/rachelorbit-or/SistemaRural-PE>

Capturas:

Compromisos

principal ▾

🔍 Todos los usuarios ▾ 📅 Todos los tiempos ▾

➤ Compromisos del 25 de septiembre de 2026

- Agregar registro de decisiones para SistemaRural-PE** Verified 6d687b7
- Agregar archivos mediante carga** Verified 7b9b77c
- Se agregan archivos del proyecto como el infome y el script en python que valida el RUC** 4e5634b

➤ Confirmado el 24 de septiembre de 2026

- Compromiso inicial** Verified f6a74ec

SistemaRural-PE Público

Alfiler ▾ Mirar 0 ▾ Tenedor 0 Estrella 0 ▾

principal ▾ 1 Sucursal 0 etiquetas

🔍 Ir al archivo

Agregar archivo ▾ **<> Código** ▾

Archivo	Descripción	Fecha
Decisiones_Grupo.md	Agregar registro de decisiones para SistemaRural-PE	Hace 1 minuto
Grupo09-UA1N1288P_EP.pdf...	Agregar archivos mediante carga	Hace 6 minutos
Grupo09-UA1N1288P_EP.pdf	Se agrega archivos del proyecto como el infome y el script ...	Hace 2 horas
LÉAME.md	Se agrega archivos del proyecto como el infome y el script ...	Hace 2 horas
validar_ruc.py	Se agrega archivos del proyecto como el infome y el script ...	Hace 2 horas

LÉAME

SistemaRural-PE

Acerca de

No se proporciona descripción, sitio web ni temas.

Léame

Actividad

☆ 0 estrellas

👁 0 espectadores

👤 0 tenedores

Comunicados

Paquetes

Colaboradores

Anexo B: Declaración de uso de Inteligencia Artificial:

Declaración de uso de Inteligencia Artificial

"Declaro que en la elaboración del presente desafío se utilizó inteligencia artificial de manera ética y bajo supervisión humana. La IA fue empleada únicamente para: (colocar X en cada ítem que corresponda)

Edición lingüística y de estilo (ej.: corrección gramatical, sugerencias de redacción).	(x)
Traducción preliminar de textos.	(x)
Apoyo en la organización o síntesis de borradores del informe, la presentación o el guion de sustentación, basados en contenidos y evidencias de campo elaborados por la autora/el autor o el equipo.	(x)
Apoyo en el procesamiento o análisis de la información recopilada en la práctica de campo (ej.: tabulación de encuestas/entrevistas, generación de gráficos a partir de datos reales, verificación de cálculos o funciones). Esto debe estar detallado en la parte del entregable correspondiente	(x)
Apoyo en la generación de diagramas, esquemas, mockups o piezas visuales como parte del desarrollo del entregable (especificando parámetros y verificando su fidelidad con la información recopilada). Esto debe estar detallado en la parte del entregable correspondiente	(x)
No se usó Inteligencia Artificial	()

La herramienta de IA utilizada fue: Claude Sonnet 5 (Anthropic).

Todo el contenido generado fue revisado, corregido y validado por la autora/el autor, asegurando precisión, integridad académica y fidelidad con los objetivos del trabajo."